

8 SQL Injection

SQL Injection (SQLi) to jedna z najczęstszych i najgroźniejszych podatności aplikacji internetowych, która polega na wstrzykiwaniu złośliwych zapytań SQL do bazy danych. Skutkiem ataku może być kradzież danych, ich modyfikacja, a nawet przejęcie pełnej kontroli nad serwerem.

Identyfikacja podatności

Aby sprawdzić, czy aplikacja jest podatna na SQL Injection, można wykonać następujące kroki:

Testowanie wejść użytkownika

Podatność najczęściej występuje w miejscach, gdzie dane wprowadzane przez użytkownika są wykorzystywane w zapytaniach SQL, np. w formularzach, parametrach URL, nagłówkach HTTP czy plikach cookie.

- Spróbuj wprowadzić znaki specjalne SQL, takie jak ' , " ; , - w polach wejściowych.
- Zastosuj typowe payloady testowe, np.:

```
' OR '1'='1'; --
" OR "1"="1"; --
' UNION SELECT NULL , NULL , NULL; --
```

Obserwowanie odpowiedzi serwera

Reakcja aplikacji na powyższe dane może wskazywać na podatność. Typowe objawy:

- Błędy bazy danych (np. `SQL syntax error`).
- Zmiana zachowania aplikacji (np. zwrócenie dodatkowych danych lub pominięcie autoryzacji).

Przykładowy atak

Załóżmy, że mamy zapytanie:

```
SELECT * FROM users WHERE username = '$username' AND password = '$password';
```

Podstawiając payload ' OR '1'='1', atakujący może ominąć uwierzytelnianie:

```
SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '' OR '1'='1';
```

Jak się chronić?

Poniżej przedstawiono najlepsze praktyki w zakresie ochrony przed SQL Injection:

- **Używanie przygotowanych zapytań (Prepared Statements)** - Zamiast dynamicznego budowania zapytań SQL, należy używać zapytań parametryzowanych. Przykład w języku Python z wykorzystaniem biblioteki `sqlite3`:

```
cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?",
               (username , password))
```

- **Walidacja i sanitacja danych wejściowych**
 - Ogranicz dane wejściowe do oczekiwanych wartości (np. używając regexów).
 - Odrzuć znaki specjalne, które mogą być używane w zapytaniach SQL.
- **Zarządzanie uprawnieniami**
 - Ogranicz uprawnienia użytkowników bazy danych.
 - Aplikacja powinna korzystać z konta bazy danych o minimalnych uprawnieniach.
- **Używanie zapór aplikacyjnych (WAF)** - Zapory aplikacyjne mogą blokować znane wzorce ataków SQL Injection.
- **Regularne testy bezpieczeństwa** - Przeprowadzaj regularne audyty i testy penetracyjne w celu wykrycia potencjalnych podatności.

Typy ataków SQL Injection

SQL Injection (SQLi) występuje w różnych formach w zależności od metod i celów atakującego. W niniejszej notatce przedstawiono najczęstsze typy ataków SQLi wraz z przykładami.

Klasyczny SQL Injection

Ten typ ataku polega na wstrzyknięciu złośliwego kodu SQL bezpośrednio do zapytania. Wykorzystuje luki w sanitacji danych wejściowych.

Przykład:

Zapytanie SQL w aplikacji:

```
SELECT * FROM users WHERE username = '$username' AND password = '$password';
```

Payload atakującego:

```
' OR '1'='1
```

Wynikowe zapytanie SQL:

```
SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '' OR '1'='1';
```

To zapytanie zwróci wszystkie rekordy w tabeli `users`, omijając uwierzytelnianie.

SQL Union Injection

Atak ten wykorzystuje operator `UNION`, aby łączyć wyniki dwóch lub więcej zapytań. Atakujący może uzyskać dodatkowe dane z bazy.

Przykład:

Zapytanie SQL w aplikacji:

```
SELECT id, name FROM users WHERE id = '$id';
```

Payload atakującego:

```
1 UNION SELECT null, database();
```

Wynikowe zapytanie SQL:

```
SELECT id, name FROM users WHERE id = '1' UNION SELECT null, database();
```

To zapytanie zwraca nazwę aktualnie używanej bazy danych.

Blind SQL Injection

Blind SQL Injection stosowany jest, gdy aplikacja nie zwraca wyników zapytania, ale zachowanie serwera wskazuje na poprawność zapytania.

Przykład:

Zapytanie SQL w aplikacji:

```
SELECT * FROM users WHERE username = '$username';
```

Payload atakującego:

```
' AND (SELECT 1 FROM dual WHERE database() = 'test') --
```

Jeśli aplikacja działa normalnie, można założyć, że baza danych nazywa się `test`. Blind SQL Injection wymaga iteracyjnego testowania.

Time-Based Blind SQL Injection

Ten rodzaj ataku opiera się na wykorzystaniu opóźnień w odpowiedziach serwera do uzyskania informacji.

Przykład:

Zapytanie SQL w aplikacji:

```
SELECT * FROM users WHERE username = '$username';
```

Payload atakującego:

```
' OR IF(SUBSTRING(database(),1,1)='t', SLEEP(5), 0); --
```

Jeśli serwer opóźnia odpowiedź o 5 sekund, atakujący wie, że pierwsza litera nazwy bazy danych to **t**. Atak wymaga iteracyjnego odpytywania.

- 9.1** Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/owasp-juiceshop-v1711), uruchom serwer za pomocą polecenia : `docker run -dp 9001:3000 mazurkatarzyna/owasp-juiceshop-v1711:latest`. Po uruchomieniu serwer działa na porcie 9001, sprawdź w przeglądarce: `http://127.0.0.1:9001`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność SQL Injection. Na serwerze znaleźć możesz stronę internetową z formularzem logowania do panelu administracyjnego. Twoim zadaniem jest zalogowanie się na konto administratora systemu, znając jedynie jego adres e-mail: `admin@juice-sh.op`. Wykorzystując atak SQL Injection, zaloguj się na konto administratora nie znając jego hasła.
- 9.2** Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/owasp-juiceshop-v1711), uruchom serwer za pomocą polecenia : `docker run -dp 9002:3000 mazurkatarzyna/owasp-juiceshop-v1711:latest`. Po uruchomieniu serwer działa na porcie 9002, sprawdź w przeglądarce: `http://127.0.0.1:9002`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność SQL Injection. Twoim zadaniem jest znalezienie poprawnych kont użytkowników w systemie. Wiedząc, że użytkownicy logują się do serwisu za pomocą adresów e-mail z końcówką `@juice-sh.op`, znajdź adresy e-mailowe wszystkich zarejestrowanych w aplikacji użytkowników.
- 9.3** Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/owasp-juiceshop-v1711), uruchom serwer za pomocą polecenia : `docker run -dp 9003:3000 mazurkatarzyna/owasp-juiceshop-v1711:latest`. Po uruchomieniu serwer działa na porcie 9003, sprawdź w przeglądarce: `http://127.0.0.1:9003`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność SQL Injection. Wiedząc, że wyszukiwarka dostępna w aplikacji zawiera podatność SQL Injection, znajdź schemę bazy danych. Podpowiedź: należy wykorzystać atak SQL Union.
- 9.4** Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/owasp-juiceshop-v1711), uruchom serwer za pomocą polecenia : `docker run -dp 9004:3000 mazurkatarzyna/owasp-juiceshop-v1711:latest`. Po uruchomieniu serwer działa na porcie 9004, sprawdź w przeglądarce: `http://127.0.0.1:9004`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność SQL Injection. Sprawdź, czy jest możliwe przeprowadzenie ataku SQL podczas przeglądania recenzji produktów w sklepie. Do czego może doprowadzić przeprowadzenie takiego ataku? Jakiego typu atak SQL Injection może zostać tu wykorzystany?
- 9.5** Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/dvwa-v110), uruchom serwer za pomocą polecenia : `docker run -dp 9095:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 9095, sprawdź w przeglądarce: `http://127.0.0.1:9095`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność SQL Injection.

Na poziomie **low security** (brak zabezpieczeń):

- Ustaw poziom zabezpieczeń na **Low**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Znajdź nazwy wszystkich użytkowników w bazie danych.
- Znajdź nazwy wszystkich kolumn w tabeli `users` w bazie.
- Znajdź hasła wszystkich użytkowników w bazie.

Na poziomie **medium security** (niewielkie zabezpieczenia):

- Ustaw poziom zabezpieczeń na **Medium**. Przeanalizuj kod strony, jakie wdrożono zabezpieczenia? Czy istnieje sposób, aby je ominąć?
- Omin zabezpieczenia i znajdź nazwy wszystkich użytkowników i ich hasła.

Na poziomie **high security** (wyższe zabezpieczenia):

- Ustaw poziom zabezpieczeń na **High**. Przeanalizuj kod strony, jakie wdrożono zabezpieczenia? Czy istnieje sposób, aby je ominąć?
- Omin zabezpieczenia i znajdź nazwy wszystkich użytkowników i ich hasła.

- 9.6** Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/dvwa-v110), uruchom serwer za pomocą polecenia : `docker run -dp 9096:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 9096, sprawdź w przeglądarce: `http://127.0.0.1:9096`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność SQL Injection.

Na poziomie **low security** (brak zabezpieczeń):

- Ustaw poziom zabezpieczeń na **Low**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Wykorzystując podatność Time Based SQL Injection, wydobądź z bazy danych wersję (numer) bazy danych.

9.7 Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/dvwa-v110), uruchom serwer za pomocą polecenia : `docker run -dp 9097:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 9007, sprawdź w przeglądarce: `http://127.0.0.1:9097`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność SQL Injection.

Na poziomie `low security` (brak zabezpieczeń):

- Ustaw poziom zabezpieczeń na `Low`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Wykorzystując podatność SQL typu Blind, znajdź nazwy tabeli w bazie, oraz nazwy pól w tabelach.

9.8 Rozwiąż zadanie **9.1** w Pythonie.

9.9 Rozwiąż zadanie **9.2** w Pythonie.